MeshSmith — Project Profile

Track 1 · Multimodal Content Creation · AMD AI DevMaster Hackathon 2026

1. Project background

Image-to-3D generation crossed a quality threshold in the last year — models like Hunyuan3D-2 produce genuinely good geometry from a single image. But almost nobody who *needs* 3D assets can use the raw output. Generated meshes are not watertight, carry no UVs and no physical scale, contain floating debris, and arrive in orientations that make FDM printing waste more plastic on support than on the part. The gap between "generated a mesh" and "shipped an asset" is manual work in Blender and a slicer — exactly the skills the people using these tools don't have.

MeshSmith closes that gap. It is a single local tool that goes from a text prompt or a photo to two finished artifacts: a **game-ready GLB** (decimated, UV-unwrapped, PBR material) and a **print-ready STL** (real millimeters, auto-oriented to minimize support, wall-thickness checked), plus a printability report that tells the user *before* they print whether the part will survive.

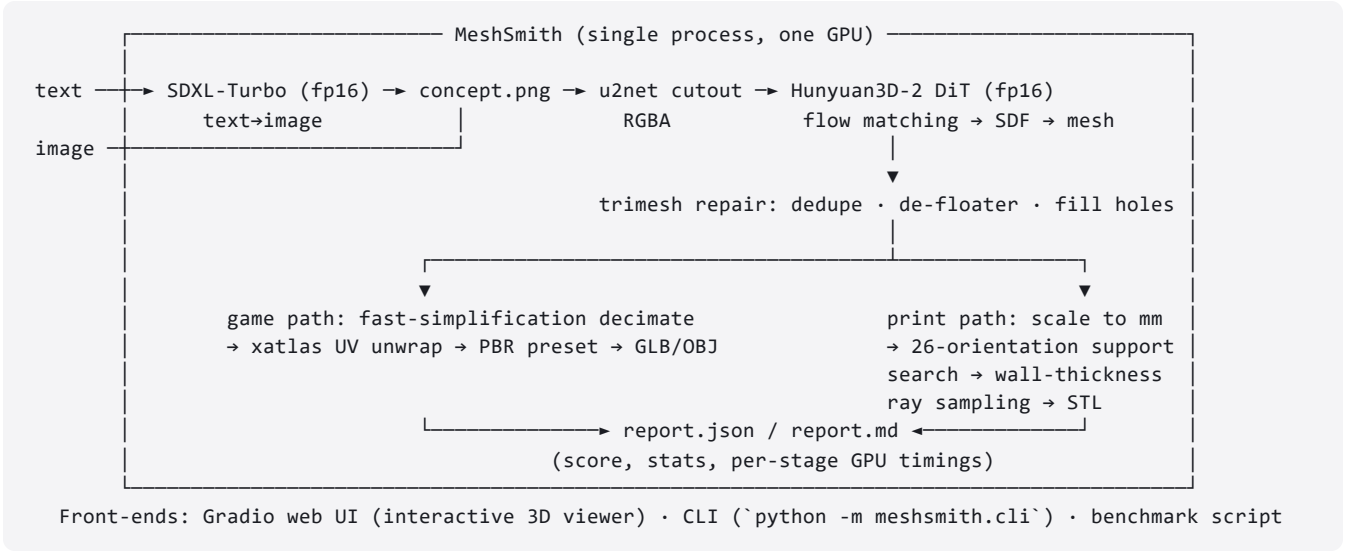
Everything runs locally. Three open models are chained on one GPU — no cloud API is involved in any core function.

2. Target users & application scenarios

User	Scenario
3D-printing hobbyists	"I need a drawer knob / figurine / bracket" → STL in minutes, no CAD or Blender skills
Indie game developers	Concept → placeholder or production props with sane topology, UVs and materials
Educators & makerspaces	Students design printable objects with natural language; the report teaches printability
Product designers	Rapid physical form studies from prompt sketches, at real scale
AR/VR creators	Fast GLB assets with correct materials for prototyping

Social value: MeshSmith removes the two gatekeeping skills of personal fabrication — CAD modeling and mesh repair. A one-line prompt becomes a physical object; the printability report replaces failed prints (and wasted plastic) with feedback before slicing. Running local means schools and makerspaces pay no per-generation API fees and keep student data on-device.

3. System architecture



Models load lazily and unload after their stage when VRAM < 30 GB, so the pipeline fits 16 GB cards; on larger GPUs both models stay resident for speed.

4. Models & algorithms

Stage	Model / algorithm	Notes
Text → image	**SDXL-Turbo** (adversarial-distilled SDXL, fp16)	1–4 steps, guidance 0; prompt auto-styled toward single-object product shots (what image→3D wants)
Subject cutout	**u2net** via rembg (ONNX, CPU)	vendor-neutral; skipped if input already has alpha
Image → 3D	**Hunyuan3D-2** flow-matching DiT + ShapeVAE (fp16); **mini-turbo** distilled variant for draft mode	*shape-only*: pure PyTorch, no custom CUDA rasterizer → portable across ROCm/CUDA
Repair	trimesh: merge/dedupe, largest-component filter, hole fill, normal fix	watertight verified before/after
Decimation	fast-simplification (quadric error metric)	to preset face budget
UV unwrap	xatlas	on the decimated mesh
Print orientation	26-direction support-volume search (overhang faces ≥45° × height prism)	rotation only — the part's physical size is preserved, so candidates are compared fairly; reports % support saved
Wall thickness	surface sampling + inward ray casting	min / 5th-percentile / % under limit

Measured effect of the orientation search, on exactly the five benchmark runs tabulated in §5 (`standard` preset, seed 7): support volume cut by **82.2%** (robot mascot), **47.5%** (fox figurine), **31.9%** (chess knight), **24.8%** (drawer knob) and **0%** (amphora). The zero is the interesting one — the generator already produced the vase upright, so no rotation beats it and the search correctly leaves the part alone rather than inventing

an improvement. Every figure above is reproduced in that run's `report.json` under `print.orientation.support_saved_pct`.

5. AMD Radeon GPU / ROCm adaptation

- **Zero vendor-specific code.** Both GPU stages run through the standard `torch.cuda` API, which ROCm PyTorch maps to HIP. Backend detection (`torch.version.hip`) is used only to *display* the platform banner and timings — code paths are identical.
- **Shape-only Hunyuan3D by design.** The texture-paint stage of Hunyuan3D-2 needs a custom CUDA rasterizer extension; MeshSmith deliberately excludes it and keeps the whole pipeline installable with `pip install -r requirements.txt` inside any `rocm/pytorch` container — no compilation step, no HIP porting.
- **fp16 everywhere** (SDXL-Turbo and Hunyuan3D DiT/VAE), sequential model residency with explicit unload + `torch.cuda.empty_cache()` on <30 GB cards.
- **Dockerfile for ROCm** (`docker/Dockerfile.rocm`) on the official `rocm/pytorch` base; runs on Radeon Cloud instances as-is.
- **Observable performance**: every run reports per-stage wall time and peak VRAM; the benchmark script reproduces the numbers below on any machine.

Benchmark (reproduce with `python scripts/benchmark.py`)

MeshSmith benchmark — Quadro RTX 8000 (NVIDIA CUDA 12.4)

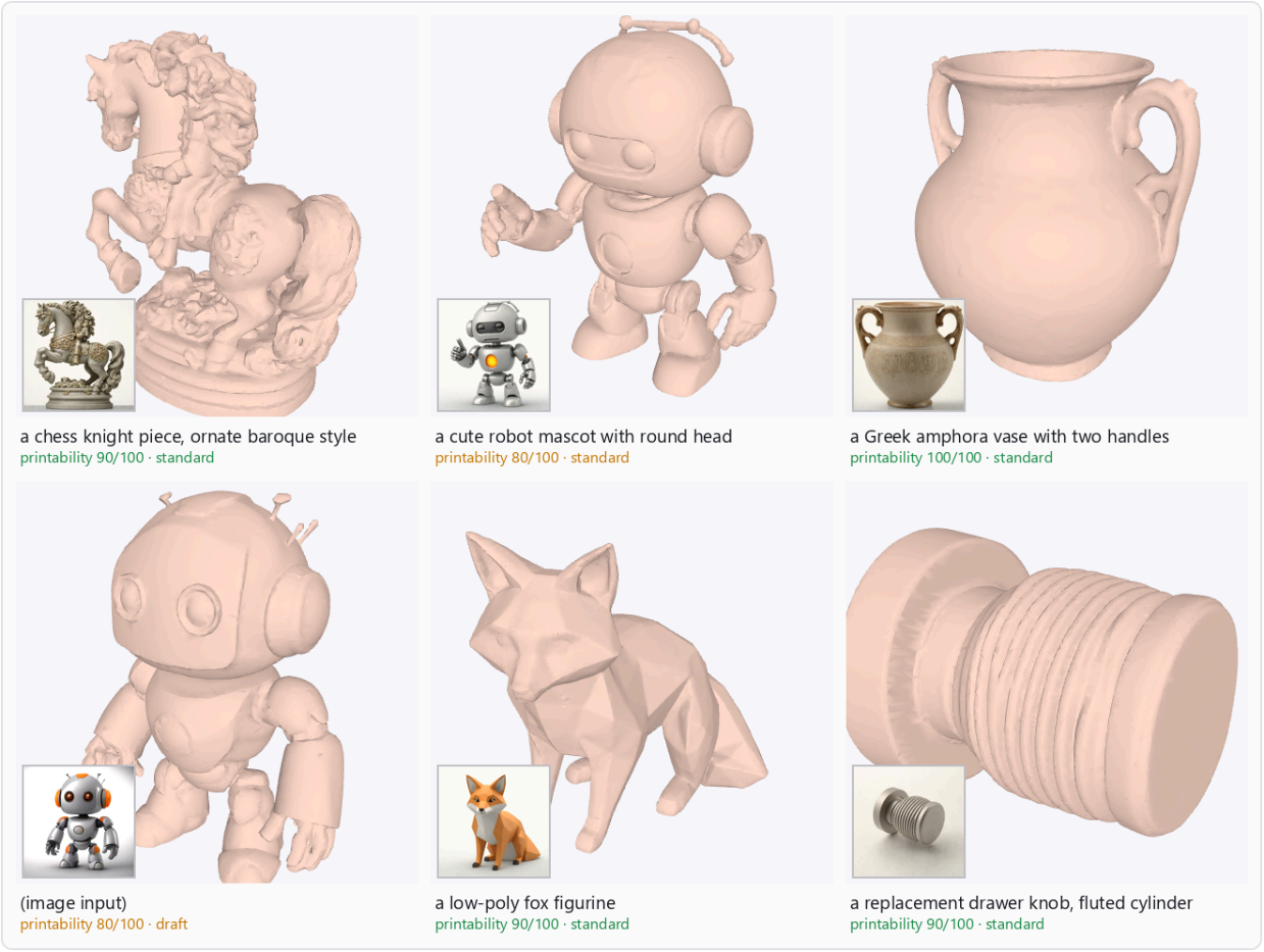
Preset: `standard` · seed 7 · torch 2.6.0+cu124

Prompt	Image (s)	3D shape (s)	Total (s)	Faces	Print score
a chess knight piece, ornate baroque style	42.6	263.5	352.6	1,107,120	90/100
a cute robot mascot with round head	2.7	196.3	240.8	866,162	80/100
a replacement drawer knob, fluted cylinder	1.2	72.4	130.2	1,466,374	90/100
a low-poly fox figurine	1.3	72.1	129.2	870,080	90/100
a Greek amphora vase with two handles	1.2	76.7	181.3	2,184,874	100/100

Average end-to-end: 206.8s per asset.

Notes: the first row includes one-time model loading (SDXL-Turbo + Hunyuan3D-2 into VRAM); warm steady state is ~2s image + ~75s shape at `standard`. The `draft` preset (Hunyuan3D-2mini-turbo) generates in ~25s warm. "Faces" is the full-detail repaired mesh; the game GLB is decimated to the preset budget.

6. Output diversity



Every asset above came out of the same pipeline with no manual mesh work: organic figures (knight, fox), a mechanical character, a vessel, and a functional hard-surface part — plus, bottom-left, the image-input path taking a photo straight to 3D. Inset thumbnails are the generated concept images. Regenerate with `scripts/build_gallery.py`.

7. Deliverables

- Source repository (MIT): pipeline package, Gradio UI, CLI, benchmark + smoke tests, ROCm Dockerfile
- This profile document, demo video (3–5 min), poster
- Example outputs: GLB/STL/report bundles for the benchmark prompt set